

Лабораторна робота №4

Аналітичні SQL-запити / OLAP

Виконав: Гасюк Д.О.

Група: ІО-41

Тема проєкту: База даних Telegram-бота для продажу цифрових товарів

Мета роботи: метою лабораторної роботи є практичне виконання аналітичних SQL-запитів до раніше створеної бази даних PostgreSQL. У роботі використовуються OLAP-запити, які дозволяють отримувати зведену статистику, групувати дані, аналізувати замовлення, платежі, товари та користувачів.

У межах лабораторної роботи було виконано:

Агрегаційні запити з використанням COUNT, SUM, AVG, MIN, MAX;

Запити з групуванням через GROUP BY;

Запити з фільтрацією груп через HAVING;

Запити з різними типами JOIN;

Запити з підзапитами у WHERE, SELECT та HAVING.

Агрегаційні запити:

Агрегаційні запити використовуються для отримання узагальненої статистики з таблиць. У роботі були використані функції COUNT, SUM, AVG, MIN, MAX, а також групування через GROUP BY.

Загальна статистика за замовленнями:

SELECT

```
COUNT(*) AS orders_count,  
SUM(total_amount) AS total_orders_amount,  
AVG(total_amount) AS average_order_amount,  
MIN(total_amount) AS min_order_amount,  
MAX(total_amount) AS max_order_amount
```

FROM bot_order;

Цей запит обчислює загальну кількість замовлень, суму всіх замовлень, середню суму замовлення, мінімальну та максимальну суму замовлення.

The screenshot shows the DBeaver SQL editor interface. The SQL editor contains the following query:

```
SELECT  
COUNT(*) AS orders_count,  
SUM(total_amount) AS total_orders_amount,  
AVG(total_amount) AS average_order_amount,  
MIN(total_amount) AS min_order_amount,  
MAX(total_amount) AS max_order_amount  
FROM bot_order;
```

The results pane displays the following data:

orders_count	total_orders_am	average_order	min_order_am	max_order_am
6	1458	243	0	450

The status bar at the bottom indicates that 1 row was received on 2025-05-26 at 12:59:45.

Групування замовлень за статусами:

SELECT

```
    status,  
    COUNT(*) AS orders_count,  
    SUM(total_amount) AS total_amount_by_status
```

FROM bot_order

GROUP BY status

ORDER BY orders_count DESC;

Цей запит групує замовлення за статусами та показує кількість замовлень і загальну суму для кожного статусу. Завдяки цьому можна побачити, скільки замовлень перебуває у статусах paid, completed, created або cancelled.

The screenshot shows the DBeaver 26.0.5 interface. The SQL editor contains the following query:

```
SELECT  
    status,  
    COUNT(*) AS orders_count,  
    SUM(total_amount) AS total_amount_by_status  
FROM bot_order  
GROUP BY status  
ORDER BY orders_count DESC;
```

The results are displayed in a table with the following data:

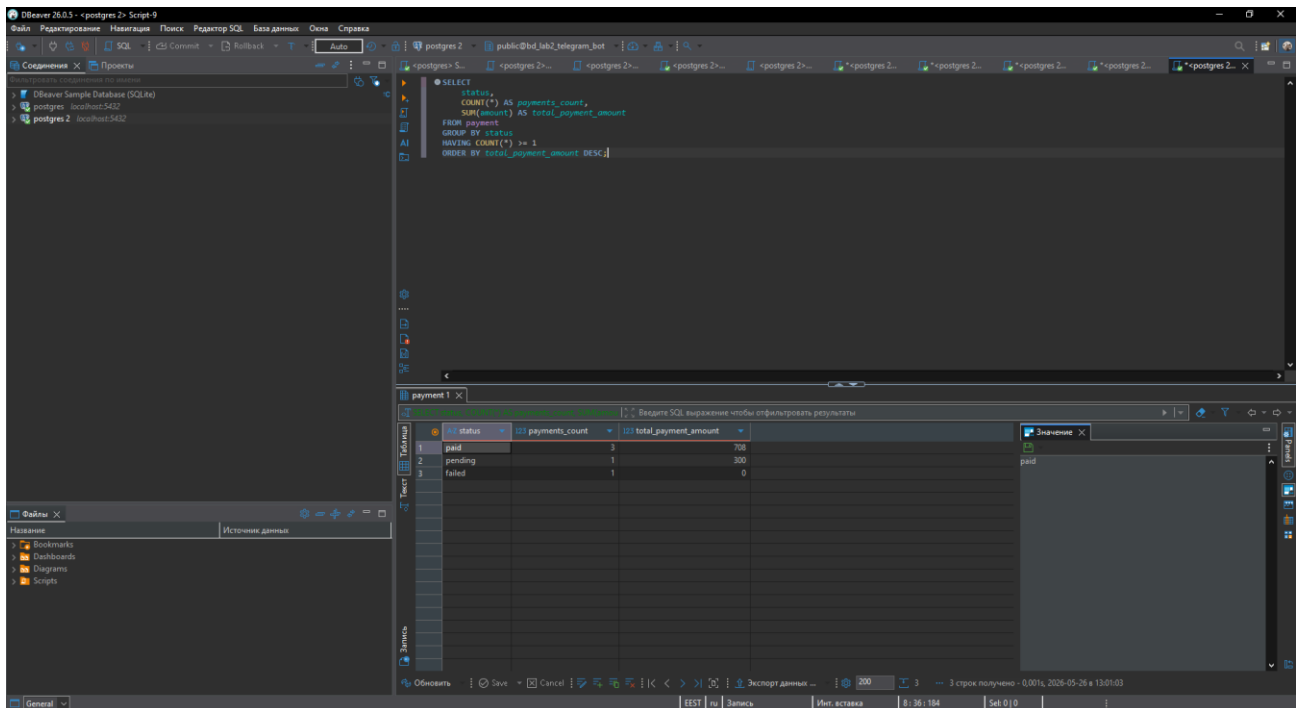
status	orders_count	total_amount_by_status
paid	3	958
completed	1	200
cancelled	1	0
created	1	300

The status bar at the bottom indicates that 4 rows were retrieved on 2025-05-26 at 13:00:35.

Групування платежів за статусами з використанням HAVING:

```
SELECT
    status,
    COUNT(*) AS payments_count,
    SUM(amount) AS total_payment_amount
FROM payment
GROUP BY status
HAVING COUNT(*) >= 1
ORDER BY total_payment_amount DESC;
```

Цей запит групує платежі за статусами та підраховує кількість платежів і загальну суму платежів у кожній групі. Умова `HAVING COUNT(*) >= 1` використовується для фільтрації груп після виконання групування.



The screenshot shows the DBeaver SQL editor interface. The SQL editor contains the following query:

```
SELECT
    status,
    COUNT(*) AS payments_count,
    SUM(amount) AS total_payment_amount
FROM payment
GROUP BY status
HAVING COUNT(*) >= 1
ORDER BY total_payment_amount DESC;
```

The results pane displays the following data:

status	payments_count	total_payment_amount
paid	3	708
pending	1	300
failed	1	0

The status bar at the bottom indicates that 3 rows were received on 2025-05-26 at 13:01:03.

Середня ціна товарів за типом товару:

```
SELECT
    product_type,
    COUNT(*) AS products_count,
    AVG(price) AS average_price,
    MIN(price) AS min_price,
    MAX(price) AS max_price
FROM product
GROUP BY product_type
ORDER BY average_price DESC;
```

Цей запит групує товари за типом і показує кількість товарів, середню, мінімальну та максимальну ціну для кожного типу товару. Він дозволяє проаналізувати, які категорії товарів мають вищу середню ціну.

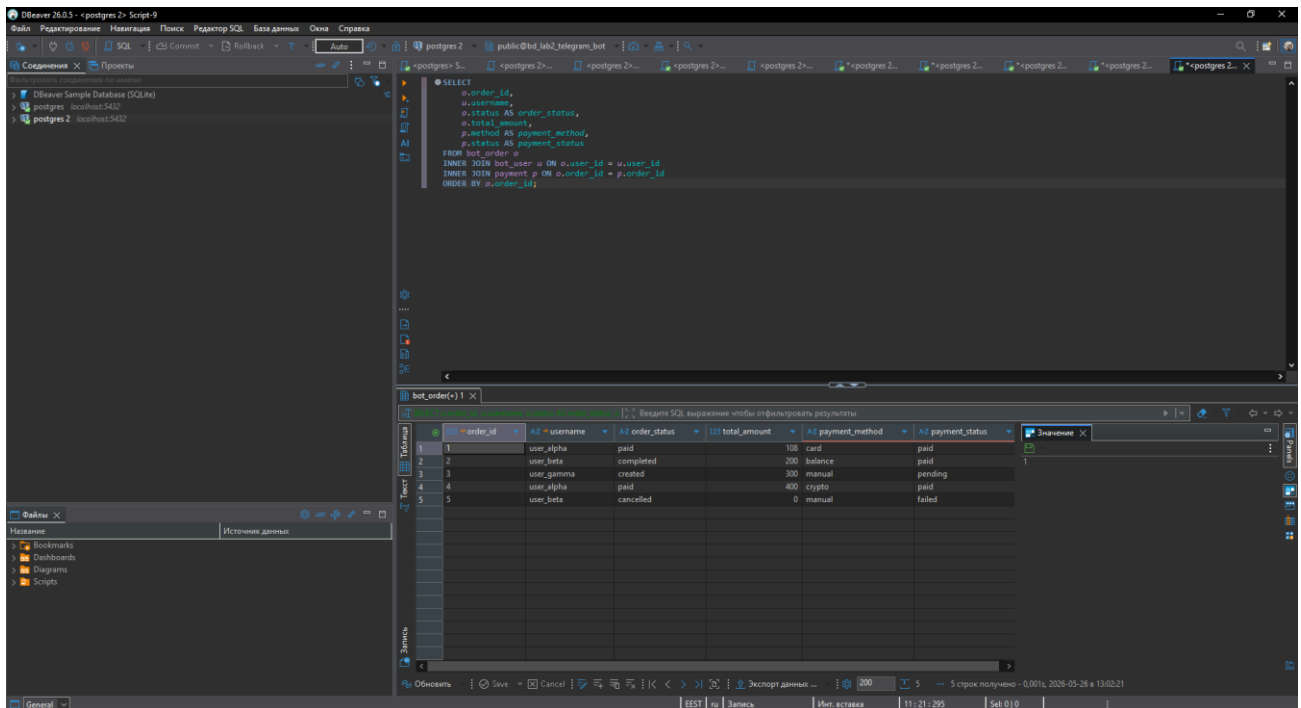
JOIN-запити:

JOIN-запити використовуються для об'єднання даних із кількох таблиць. У роботі були виконані запити з використанням INNER JOIN, LEFT JOIN та RIGHT JOIN.

INNER JOIN замовлень, користувачів і платежів:

```
SELECT
    o.order_id,
    u.username,
    o.status AS order_status,
    o.total_amount,
    p.method AS payment_method,
    p.status AS payment_status
FROM bot_order o
INNER JOIN bot_user u ON o.user_id = u.user_id
INNER JOIN payment p ON o.order_id = p.order_id
ORDER BY o.order_id;
```

Цей запит об'єднує таблиці bot_order, bot_user та payment. Він показує тільки ті замовлення, для яких існує відповідний користувач і відповідний платіж.



LEFT JOIN для перегляду всіх замовлень:

```
SELECT
  o.order_id,
  u.username,
  o.status AS order_status,
  o.total_amount,
  p.payment_id,
  p.status AS payment_status,
  p.paid_at
FROM bot_order o
LEFT JOIN bot_user u ON o.user_id = u.user_id
LEFT JOIN payment p ON o.order_id = p.order_id
ORDER BY o.order_id;
```

Цей запит виводить усі замовлення з таблиці bot_order. Якщо для певного замовлення немає платежу, то поля з таблиці payment мають значення NULL. Це дозволяє знайти замовлення, які ще не мають відповідного платежу.

The screenshot shows the DBeaver SQL editor interface. The top pane contains a SQL query that joins the bot_order and payment tables. The bottom pane displays the results of the query in a table format.

```
SELECT
  o.order_id,
  o.username,
  o.status AS order_status,
  o.total_amount,
  p.payment_id,
  p.status AS payment_status,
  p.total_at
FROM bot_order o
LEFT JOIN bot_user u ON o.user_id = u.user_id
LEFT JOIN payment p ON o.order_id = p.order_id
ORDER BY o.order_id
```

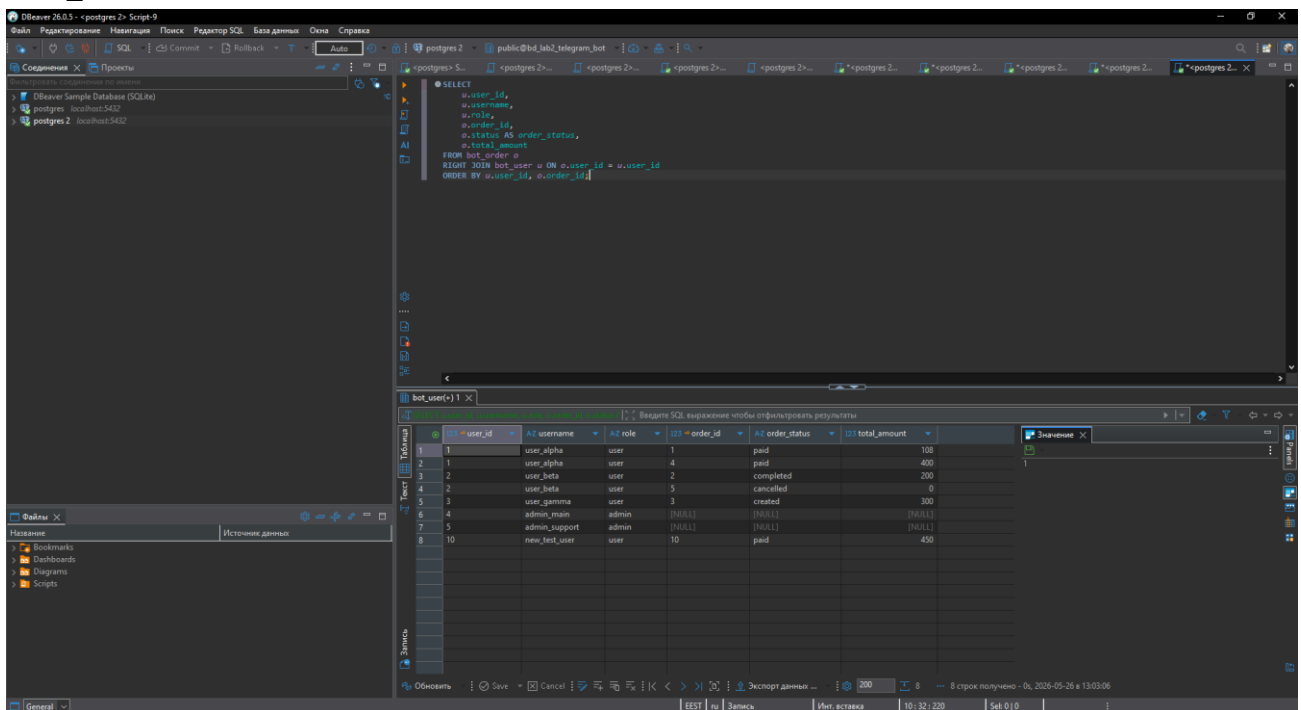
order_id	username	order_status	total_amount	payment_id	payment_status
1	user_alpha	paid	108	1	paid
2	user_beta	completed	200	2	paid
3	user_gamma	created	300	3	pending
4	user_alpha	paid	400	4	paid
5	user_beta	cancelled	0	5	failed
10	new_test_user	paid	450	[NULL]	[NULL]

The status bar at the bottom indicates that 6 rows were received from the database at 2025-05-26 13:02:40.

RIGHT JOIN для перегляду всіх користувачів:

```
SELECT
    u.user_id,
    u.username,
    u.role,
    o.order_id,
    o.status AS order_status,
    o.total_amount
FROM bot_order o
RIGHT JOIN bot_user u ON o.user_id = u.user_id
ORDER BY u.user_id, o.order_id;
```

Цей запит виводить усіх користувачів із таблиці bot_user, навіть якщо вони не мають замовлень. Якщо користувач не створював замовлення, поля order_id, order_status та total_amount мають значення NULL.



The screenshot shows the DBeaver SQL editor with a query window and a results window. The query is a RIGHT JOIN between bot_order and bot_user tables. The results window displays 8 rows of data, including users with orders and users without orders (where order fields are NULL).

	user_id	username	role	order_id	order_status	total_amount
1	1	user_alpha	user	1	paid	100
2	1	user_alpha	user	4	paid	400
3	2	user_beta	user	2	completed	200
4	2	user_beta	user	5	cancelled	0
5	3	user_gamma	user	3	created	300
6	4	admin_main	admin	[NULL]	[NULL]	[NULL]
7	5	admin_support	admin	[NULL]	[NULL]	[NULL]
8	10	new_test_user	user	10	paid	450

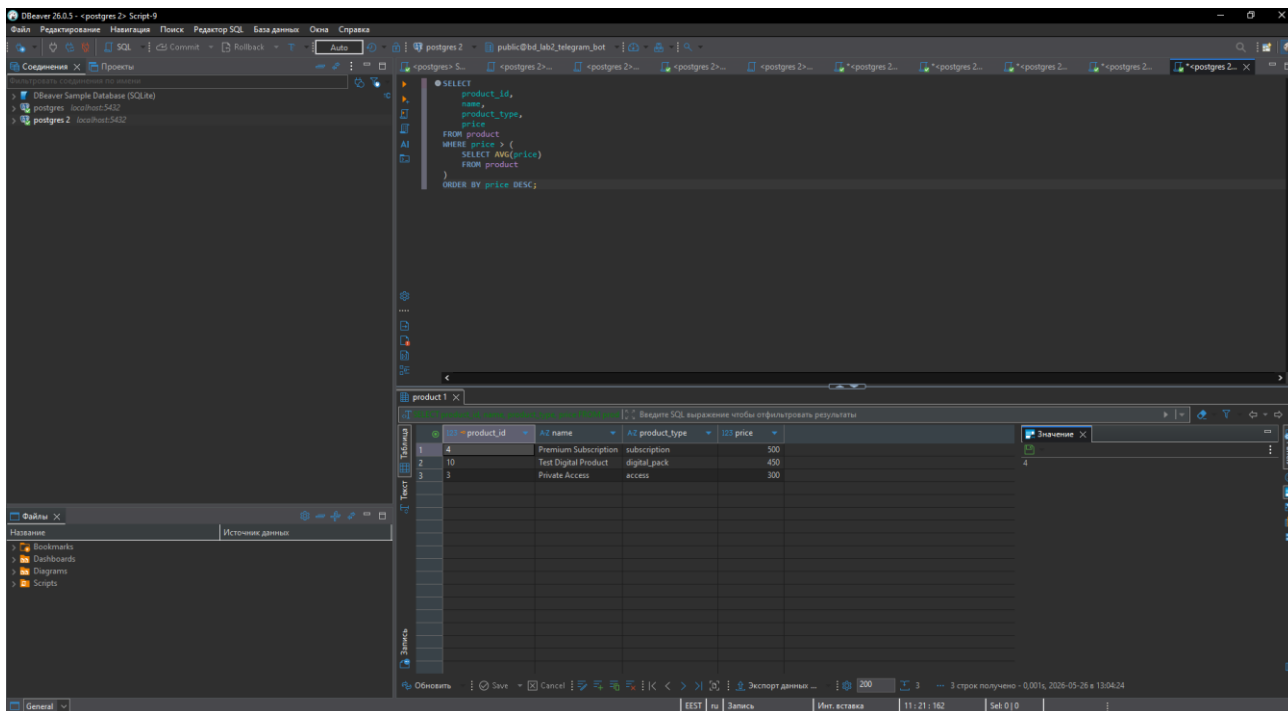
Запити з підзапитами:

Підзапити дозволяють використовувати результат одного запиту всередині іншого. У роботі були виконані підзапити у WHERE, SELECT та HAVING.

Підзапит у WHERE: товари з ціною вище середньої:

```
SELECT
    product_id,
    name,
    product_type,
    price
FROM product
WHERE price > (
    SELECT AVG(price)
    FROM product
)
ORDER BY price DESC;
```

Цей запит знаходить товари, ціна яких більша за середню ціну всіх товарів у таблиці product. Підзапит обчислює середню ціну, а основний запит вибирає товари, які перевищують це значення.



The screenshot shows the DBeaver 26.0.5 interface. The SQL editor contains the following query:

```
SELECT
    product_id,
    name,
    product_type,
    price
FROM product
WHERE price > (
    SELECT AVG(price)
    FROM product
)
ORDER BY price DESC;
```

The results pane displays the following data:

product_id	name	product_type	price
4	Premium Subscription	subscription	500
10	Test Digital Product	digital_pack	450
3	Private Access	access	300

The status bar at the bottom indicates that 3 rows were retrieved on 2025-05-26 at 13:04:24.

Підзапит у SELECT: кількість замовлень для кожного користувача:

SELECT

```
    u.user_id,  
    u.username,  
    u.role,  
    (  
        SELECT COUNT(*)  
        FROM bot_order o  
        WHERE o.user_id = u.user_id  
    ) AS orders_count
```

FROM bot_user u

ORDER BY orders_count DESC;

Цей запит показує список користувачів і кількість замовлень кожного користувача. Підзапит у частині SELECT підраховує кількість замовлень для кожного окремого користувача.

The screenshot shows the DBeaver 26.0.5 interface. The SQL editor contains the following query:

```
SELECT  
    u.user_id,  
    u.username,  
    u.role,  
    (  
        SELECT COUNT(*)  
        FROM bot_order o  
        WHERE o.user_id = u.user_id  
    ) AS orders_count  
FROM bot_user u  
ORDER BY orders_count DESC;
```

The results are displayed in a table with the following columns: user_id, username, role, and orders_count. The data is sorted by orders_count in descending order.

user_id	username	role	orders_count
1	user_alpha	user	2
2	user_beta	user	2
3	user_gamma	user	1
4	new_test_user	user	1
5	admin_main	admin	0
6	admin_support	admin	0

The status bar at the bottom indicates that 6 rows were retrieved.

Підзапит у HAVING: користувачі із сумою замовлень вище середньої:

```
SELECT
    u.username,
    COUNT(o.order_id) AS orders_count,
    SUM(o.total_amount) AS total_spent
FROM bot_user u
JOIN bot_order o ON u.user_id = o.user_id
GROUP BY u.username
HAVING SUM(o.total_amount) > (
    SELECT AVG(total_amount)
    FROM bot_order
)
ORDER BY total_spent DESC;
```

Цей запит групує замовлення за користувачами та обчислює загальну суму витрат кожного користувача. Умова HAVING залишає тільки тих користувачів, у яких загальна сума замовлень більша за середню суму одного замовлення.

The screenshot shows the DBeaver 26.0.5 interface. The SQL editor contains the following query:

```
SELECT
    u.username,
    COUNT(o.order_id) AS orders_count,
    SUM(o.total_amount) AS total_spent
FROM bot_user u
JOIN bot_order o ON u.user_id = o.user_id
GROUP BY u.username
HAVING SUM(o.total_amount) > (
    SELECT AVG(total_amount)
    FROM bot_order
)
ORDER BY total_spent DESC;
```

The results pane shows the following data:

username	orders_count	total_spent
user_alpha	2	508
new_bot_user	1	450
user_gamma	1	300

The status bar at the bottom indicates that 3 rows were retrieved in 0.002s (0.001s per row) on 2025-05-26 at 13:04:55.

Короткий опис виконаних операцій:

У межах лабораторної роботи були виконані такі типи аналітичних SQL-запитів:

Тип запиту	Призначення
Агрегаційні запити	Обчислення загальної кількості, суми, середнього, мінімального та максимального значення
GROUP BY	Групування замовлень, платежів і товарів за певними ознаками
HAVING	Фільтрація результатів після групування
INNER JOIN	Вибір записів, які мають відповідники в усіх об'єднаних таблицях
LEFT JOIN	Виведення всіх записів з лівої таблиці, навіть якщо відповідників у правій таблиці немає
RIGHT JOIN	Виведення всіх записів з правої таблиці, навіть якщо відповідників у лівій таблиці немає
Підзапити	Використання результату одного запиту всередині іншого запиту

Висновки:

У результаті виконання лабораторної роботи було реалізовано аналітичні SQL-запити до бази даних Telegram-бота для продажу цифрових товарів. Було використано агрегатні функції COUNT, SUM, AVG, MIN, MAX, групування через GROUP BY, фільтрацію згрупованих результатів через HAVING, а також різні типи об'єднань таблиць.

За допомогою INNER JOIN, LEFT JOIN та RIGHT JOIN було отримано об'єднану інформацію про користувачів, замовлення та платежі. Також були виконані запити з підзапитами, які дозволили знайти товари з ціною вище середньої, підрахувати кількість замовлень для кожного користувача та визначити користувачів із сумою замовлень вище середньої. Отримані результати підтвердили, що створена база даних коректно підтримує аналітичні запити. Дані можна групувати, об'єднувати з кількох таблиць, фільтрувати за агрегованими умовами та аналізувати за допомогою підзапитів.